



# SIMATS Online Programming Lab

## JAVA PROGRAMMING



KADIRI ASMITHA  
192425038RD

### Multithreading Practice - CO3

[← Back](#)

#### QUESTIONS

**Q1**

Multiple Threads Using a Single Class  
10 marks

**Q2**

Multiple thread classes  
20 marks

**Q3**

Thread Synchronization  
10 marks



3/3 answered

#### Q2 Multiple thread classes

20 marks

Question:

Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.

FactorialThread → Find the factorial of the given number.

ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.

Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number

Factorial: 7! = 5040

Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

## Your Code

```
1 import java.util.Scanner;
2
3 public class PrimeThread extends Thread {
4
5     int n;
6
7     PrimeThread(int n) {
8         this.n = n;
9     }
10
11     public void Prime(int n) {
12         int count = 0;
13
14         for (int i = 1; i <= n; i++) {
15             if (n % i == 0) {
16                 count++;
17             }
18         }
19
20         if (count == 2)
21             System.out.println("Prime: " + n);
22         else
23             System.out.println("Prime: " + n);
24     }
25
26     public void run() {
27         Prime(n);
28     }
29
30     public static void main(String[] args) {
31
32         Scanner sc = new Scanner(System.in);
33
34         int n = sc.hasNextInt() ? sc.nextInt() : 0;
35
36         PrimeThread t1 = new PrimeThread(n);
37         FactorialThread t2 = new FactorialThread(n);
38         ReverseThread t3 = new ReverseThread(n);
39
40         t1.start();
41         t1.join();
42
43         t2.start();
44         t2.join();
45
46         t3.start();
```

```
47         t3.join();
48     }
49 }
50
51 class FactorialThread extends Thread {
52
53     int n;
54
55     FactorialThread(int n) {
56         this.n = n;
57     }
58
59     public void Factorial(int n) {
60         int fact = 1;
61
62         for (int i = 1; i <= n; i++) {
63             fact = fact * i;
64         }
65
66         System.out.println("Factorial: " + fact);
67     }
68
69     public void run() {
70         Factorial(n);
71     }
72 }
73
74 class ReverseThread extends Thread {
75
76     int n;
77
78     ReverseThread(int n) {
79         this.n = n;
80     }
81
82     public void Reverse(int n) {
83         int temp = n;
84         int rev = 0;
85
86         for (; n > 0; n = n / 10) {
87             int digit = n % 10;
88             rev = rev * 10 + digit;
89         }
90
91         System.out.println("Reverse: " + rev);
92     }
93
94     public void run() {
```

```
95         Reverse(n) ;  
96     }  
97 }
```



### Input

7

### Output

```
Prime: 7 is a prime number  
Factorial: 7! = 5040  
Reverse: Reverse of 7 = 7
```

### Visible Test Cases

Test Case 1



Test Case 2



Test Case 3



20.00 / 20 marks

✓ Submitted